SHOPSPHERE ENGINEERING
ARCHITECTURE DOCUMENTATION

ShopSphere Architecture Overview

Core services, checkout payment flow, retry behaviour, data stores and monitoring

Document number	SS-ARCH-001
Version	1.3
Last updated	2026-09-02
Owner	Platform Team
Audience	New developers joining ShopSphere
Status	Current
Related documents	Payment API Specification v2.0; Payment Service Deployment Runbook v3.1

Fictional document. ShopSphere, its services, people and data are synthetic. This document is part of the TraceIQ training dataset and is not documentation for any real system.

Contents

1. Purpose . . . 3
1.1 Scope . . . 3
1.2 Conventions . . . 3

2. Core Services . . . 3
2.1 System Diagram . . . 4
2.2 Service Details . . . 4

3. Checkout Payment Flow . . . 5
3.1 Sequence Diagram . . . 6
3.2 Step-by-Step Reference . . . 6
3.3 The Two Payment Flows . . . 6
3.4 Order Outcome . . . 7

4. Retry Behaviour Between Services . . . 7
4.1 Timing of a Payment That Fails Every Attempt . . . 7
4.2 Idempotency . . . 8

5. Data Stores . . . 8

6. Monitoring . . . 8

7. Request Handling Inside payment-service . . . 9

8. Logging and Tracing . . . 10

9. Deployment Model . . . 10

10. Glossary . . . 11

11. Related Documents and Revision History . . . 11

1. Purpose

This document explains how the ShopSphere e-commerce platform is built. It covers the core services, how a checkout flows through them, and how failures between services are handled. It is written for new developers.

1.1 Scope

The document describes the six core components of ShopSphere: four services (api-gateway, order-service, payment-service and inventory-service) and two data stores (postgres and redis). For each component it states what the component does, which components it calls, and which data it owns.

The exact request and response format of the payment endpoint is not repeated here. It is defined in the Payment API Specification v2.0. Deployment and rollback procedures for payment-service are defined in the Payment Service Deployment Runbook v3.1.

1.2 Conventions

Convention	Meaning in this document
Versions	Service versions are the versions running in production in September 2026.
Caller and callee	"A calls B" means A sends requests to B. In diagrams, an arrow points from the caller to the callee.
Time	All times are UTC. Durations are in milliseconds (ms) unless stated otherwise.
Latency percentiles	p50, p95 and p99 are the 50th, 95th and 99th percentile of request latency.

2. Core Services

ShopSphere has four services and two data stores. The table below lists each component with its production version in September 2026.

Service	Version (2026-09)	What it does	Talks to
`api-gateway`	1.9.2	Public entry point. Routes /checkout and /orders.	order-service, redis
`order-service`	5.7.0	Creates orders. Reserves stock, then asks for payment.	inventory-service, payment-service, postgres
`payment-service`	2.3.1	Validates payment requests and records payments.	postgres, redis
`inventory-service`	3.2.0	Reserves and releases stock.	postgres
`postgres`	15.6	Main database (orders, payments, inventory tables).	-
`redis`	7.2.4	Session cache and payment idempotency-key cache.	-

Table 2-1. Core components of ShopSphere.

2.1 System Diagram

The diagram shows every caller-to-callee relationship between the six components. Blue boxes are services; orange boxes are data stores. Labels describe what each call is used for.

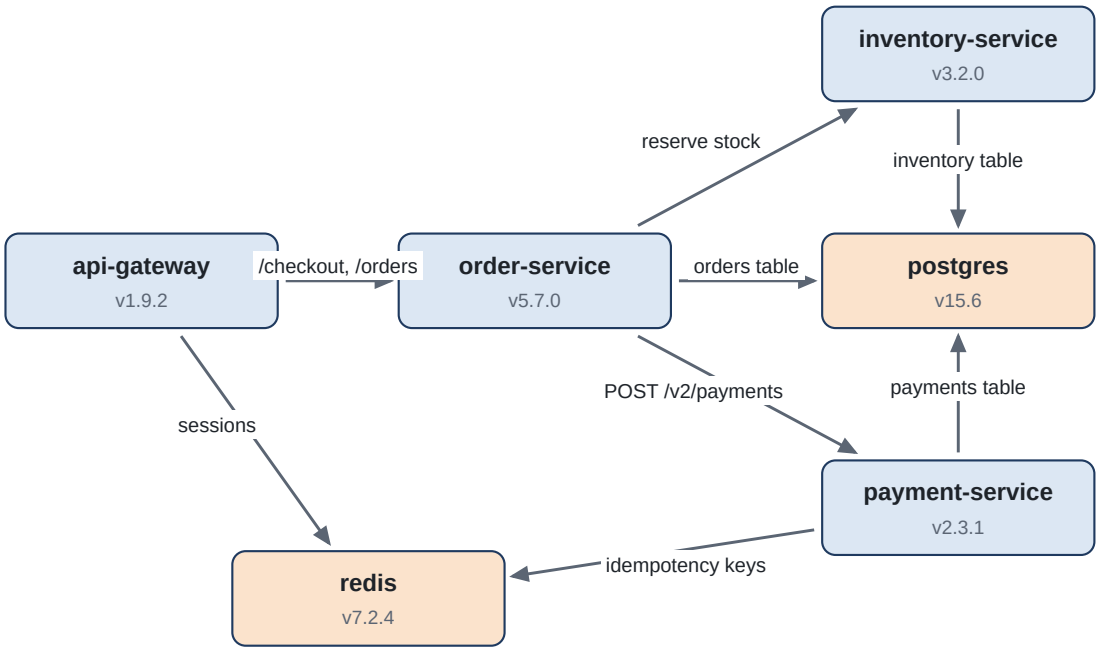

Figure 2-1. ShopSphere components and their dependencies (arrow points from caller to callee).

The dependency graph has no cycles: no service calls back into a service that called it. Customers only reach ShopSphere through api-gateway. No customer request reaches postgres or redis directly.

2.2 Service Details

api-gateway

Attribute	Value
Version (2026-09)	1.9.2
Responsibility	Public entry point for all customer traffic. Routes /checkout and /orders to order-service.
Calls	order-service, redis
Called by	Customers
Data owned	Web sessions (stored in redis)

order-service

Attribute	Value
Version (2026-09)	5.7.0
Responsibility	Creates orders. For each order it reserves stock with inventory-service, then asks payment-service for payment, then records the result.
Calls	inventory-service, payment-service, postgres
Called by	api-gateway
Data owned	orders table (postgres)

payment-service

Attribute	Value
Version (2026-09)	2.3.1
Responsibility	Validates payment requests and records payments. Exposes POST /v2/payments. Validates the request body, checks the idempotency key in redis, and writes the payment to postgres.
Calls	postgres, redis
Called by	order-service
Data owned	payments table (postgres); idempotency keys (redis)

inventory-service

Attribute	Value
Version (2026-09)	3.2.0
Responsibility	Reserves and releases stock.
Calls	postgres
Called by	order-service
Data owned	inventory table (postgres)

Data stores

Attribute	postgres	redis
Version	15.6	7.2.4
Role	Main database (orders, payments, inventory tables).	Session cache and payment idempotency-key cache.
Used by	order-service, payment-service, inventory-service	api-gateway, payment-service

3. Checkout Payment Flow

1) The customer submits checkout to api-gateway. 2) api-gateway forwards it to order-service. 3) order-service reserves stock with inventory-service. 4) order-service sends POST /v2/payments to payment-service. 5) payment-service validates the request body, checks the idempotency key in redis, and writes the payment to postgres. 6) order-service marks the order PAID or PAYMENT_FAILED.

order-service sends payments from two flows: the **checkout** flow (customer orders) and the **subscription-renewal** flow (scheduled monthly renewals). The two flows build the payment request with separate code.

3.1 Sequence Diagram

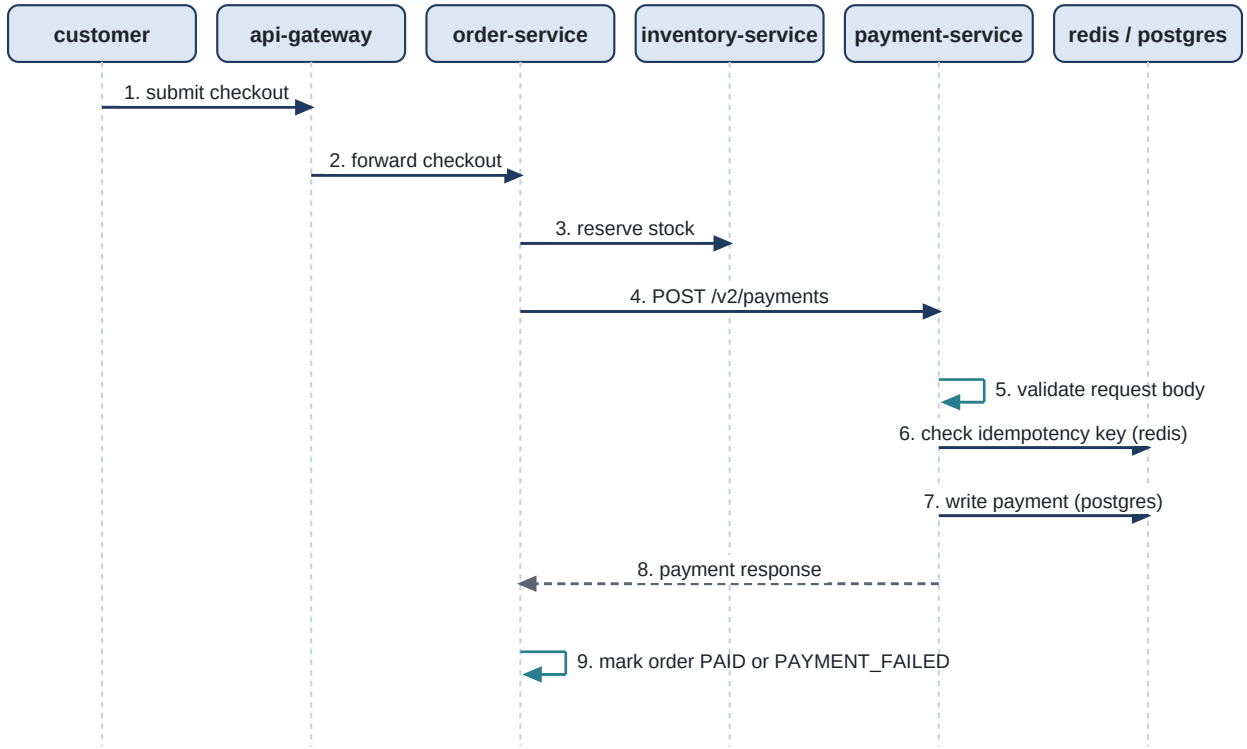

Figure 3-1. Checkout payment flow, steps 1 to 6 (dashed arrow = response).

3.2 Step-by-Step Reference

Step	Caller	Callee	Action
1	`customer`	`api-gateway`	Customer submits checkout.
2	`api-gateway`	`order-service`	api-gateway forwards the checkout to order-service.
3	`order-service`	`inventory-service`	order-service reserves stock.
4	`order-service`	`payment-service`	order-service sends POST /v2/payments.
5	`payment-service`	`redis, postgres`	payment-service validates the request body, checks the idempotency key in redis, and writes the payment to postgres.
6	`order-service`	`postgres`	order-service marks the order PAID or PAYMENT_FAILED.

Table 3-1. Checkout steps.

3.3 The Two Payment Flows

Both flows end in the same call, POST /v2/payments to payment-service, and both use the same retry behaviour (Section 4). They differ in what triggers them and in the code that builds the request.

Flow	Triggered by	Request built by	Endpoint
`checkout`	A customer order submitted through api-gateway.	Checkout code in order-service.	`POST /v2/payments`
`subscription-renewal`	Scheduled monthly renewals.	Renewal code in order-service (separate from the checkout code).	`POST /v2/payments`

Table 3-2. Payment flows in order-service.

3.4 Order Outcome

When the payment step finishes, order-service records one of two outcomes on the order:

- **PAID** - payment-service accepted and recorded the payment.
- **PAYMENT_FAILED** - payment-service did not return a successful response, including after all retries described in Section 4.

4. Retry Behaviour Between Services

order-service retries a failed payment call up to **3 times** with exponential backoff of **200 ms, 400 ms and 800 ms**. A call is treated as failed when payment-service returns any non-2xx status. Retries reuse the same idempotency key, so a retried payment can never be charged twice. Because of the waits, an order whose payment fails on every attempt takes roughly 1.4 seconds or more before order-service gives up.

Setting	Value
`max_retries`	3
`backoff`	200 ms, 400 ms, 800 ms
`retry_on`	any non-2xx response
`timeout per attempt`	3000 ms

Table 4-1. order-service retry policy for payment calls.

4.1 Timing of a Payment That Fails Every Attempt

One order can therefore produce up to four calls to payment-service: the first attempt plus three retries. Between attempts, order-service waits 200 ms, then 400 ms, then 800 ms.

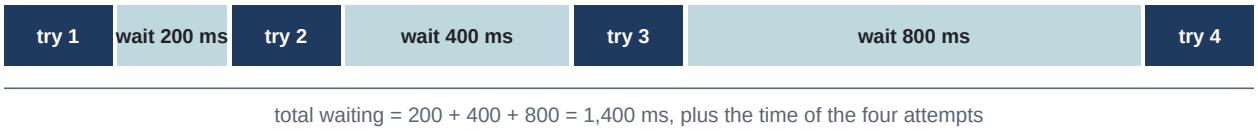

Figure 4-1. Four attempts and three backoff waits (wait widths to scale; attempt boxes are fixed width).

Event	Wait before it	Waiting so far
Attempt 1	0 ms	0 ms
Retry 1 (attempt 2)	200 ms	200 ms
Retry 2 (attempt 3)	400 ms	600 ms
Retry 3 (attempt 4)	800 ms	1,400 ms
Give up, mark PAYMENT_FAILED	-	1,400 ms + time spent in the four attempts

Table 4-2. Cumulative waiting time when every attempt fails.

If each failed attempt returns quickly, the order takes slightly more than 1.4 seconds. If each attempt runs until the 3000 ms per-attempt timeout, the worst case is 4 x 3000 ms + 1,400 ms = 13,400 ms.

4.2 Idempotency

Every payment request carries an idempotency key that is unique per order. Retries reuse the same key. payment-service checks the key in redis before writing a payment, and keeps idempotency keys in redis for 24 hours. Because of this, a retried payment can never be charged twice.

> **Effect on request counts.** Retries multiply the number of calls payment-service receives. When most attempts fail, payment-service can receive up to four requests for every order that order-service processes.

5. Data Stores

postgres holds the orders, payments and inventory tables. redis holds web sessions (used by api-gateway) and payment idempotency keys (used by payment-service, kept for 24 hours). Neither data store is called directly by the customer.

Store	Data	Written by	Retention / notes
`postgres`	orders table	`order-service`	Order record and outcome (PAID / PAYMENT_FAILED).
`postgres`	payments table	`payment-service`	One row per recorded payment.
`postgres`	inventory table	`inventory-service`	Stock levels and reservations.
`redis`	web sessions	`api-gateway`	Session cache.
`redis`	payment idempotency keys	`payment-service`	Kept for 24 hours.

Table 5-1. What each data store holds.

6. Monitoring

Each service exports request count, error count and latency percentiles (p50, p95, p99) every 5 minutes. The alert PaymentFailureRateHigh fires when the payment failure rate stays above 5% for 5 minutes.

Metric	Unit	Description
request count	requests per 5 minutes	Number of requests handled in the 5-minute bucket.
error count	requests per 5 minutes	Number of requests that failed in the bucket.
latency p50 / p95 / p99	ms	Latency percentiles for requests in the bucket.

Table 6-1. Metrics exported by every service.

Alert	Condition	Window
`PaymentFailureRateHigh`	Payment failure rate above 5%	5 minutes

Table 6-2. Payment alert.

Because order-service retries failed payment calls (Section 4), the failure rate measured on payment-service counts every attempt, while the failure rate of orders counts each order once. The two numbers are not the same when retries happen.

7. Request Handling Inside payment-service

payment-service handles every POST /v2/payments request in the same order. Each stage either passes the request on or stops it with an error code from the Payment API Specification v2.0.

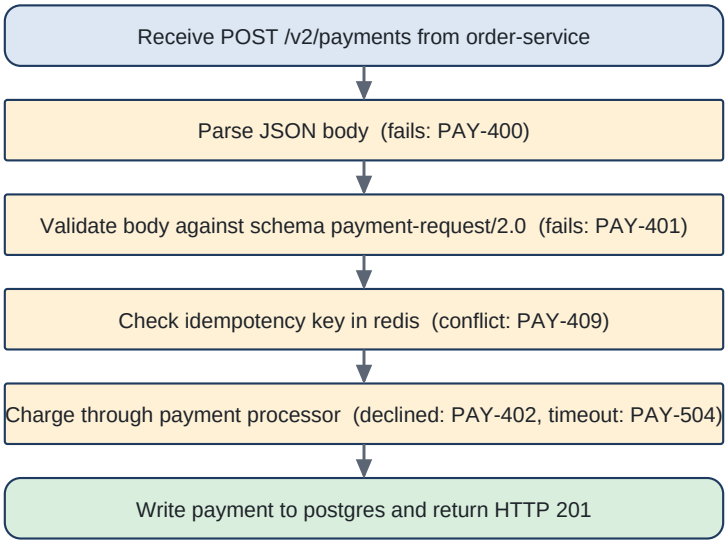

Figure 7-1. Stages of a payment request inside payment-service.

Stage	Component	Error if it fails	HTTP status
Parse body	payment-service	`PAY-400`	400
Validate fields	payment-service, validation library schemaguard 2.8.4	`PAY-401`	422
Idempotency check	payment-service + redis	`PAY-409`	409
Charge	payment-service + payment processor	`PAY-402 (declined) / PAY-504 (timeout)`	402 / 504
Record	payment-service + postgres	-	201 on success

Table 7-1. Stages, components and error codes (error codes as defined in Payment API Specification v2.0).

Validation uses the schema payment-request/2.0. The validation library in payment-service 2.3.1 is schemaguard 2.8.4. The list of fields and which of them are required is defined in the Payment API Specification, not in this document.

8. Logging and Tracing

Every service writes structured JSON log records. A trace ID is created when a request enters api-gateway and is passed to every service that handles the same request. Searching the logs for one trace ID shows the whole path of a request, including every payment retry.

Field	Meaning
`log_id`	Unique ID of the log record.
`timestamp`	UTC time in ISO 8601 format, with milliseconds.
`severity`	DEBUG, INFO, WARN or ERROR.
`service, version`	Which service and which version wrote the record.
`trace_id`	Shared ID for all records that belong to one request path.
`request_id`	ID of the payment request, when there is one.
`error_code, http_status`	Application error code and HTTP status, when the record is about an error.
`message`	Human-readable description.

Table 8-1. Fields of a structured log record.

Example log record (illustrative values)

```
{
  "log_id": "LOG-0001",
  "timestamp": "2026-09-01T12:00:00.000Z",
  "severity": "INFO",
  "service": "payment-service",
  "version": "2.3.1",
  "trace_id": "trc-000001",
  "request_id": "req-0001",
  "error_code": null,
  "http_status": 201,
  "message": "Payment captured"
}
```

Metrics (Section 6) show how many requests fail; log records show which requests failed and why. Use both together when investigating a problem.

9. Deployment Model

Services are deployed through the deploy pipeline. payment-service runs as four instances and is updated with a rolling deploy, one instance after another. After a deploy, the health endpoint GET /healthz is checked on every instance.

Important. GET /healthz only shows that the process is up. It does not send payment requests. Post-deploy checks for payment-service are described in the Payment Service Deployment Runbook v3.1.

A rollback is done by running the deploy pipeline again with the previous version. The full procedure, including approvals, is in the runbook.

10. Glossary

Term	Meaning
Backoff	The wait between one failed attempt and the next retry (200, 400, 800 ms).
Caller / callee	The service that sends a request / the service that receives it.
Idempotency key	A value unique per order that lets payment-service detect a repeated payment request and never charge twice.
Non-2xx response	Any HTTP status outside 200-299. order-service treats it as a failed payment call.
PAID / PAYMENT_FAILED	The two outcomes order-service records after the payment step.
p50 / p95 / p99	Latency percentiles: half, 95% and 99% of requests are faster than this value.
Trace ID	An ID shared by every log record that belongs to one request path.
Rolling deploy	Updating instances one at a time so the service keeps running.

11. Related Documents and Revision History

Document	Version	What it covers
Payment API Specification	2.0	Request fields, deprecated fields and error codes of POST /v2/payments.
Payment Service Deployment Runbook	3.1	Pre-deploy checks, diagnostics and rollback for payment-service.

Version	Date	Change
1.3	2026-09-02	Current version (this document).